SETUP GUIDE

Monitor Couchbase Capella with AppDynamics

See every call your Java app makes to Capella, and report any Capella metric to AppDynamics so you can alert on it. CPU is the first test metric. Written for a first-time setup, start to finish.

Java application · Couchbase Capella · AppDynamics SaaS

Requirements · 1 Connect to Capella · 2 App view · 3 Report metrics · 4 Alert · 5 Test · 6 More metrics · Troubleshooting

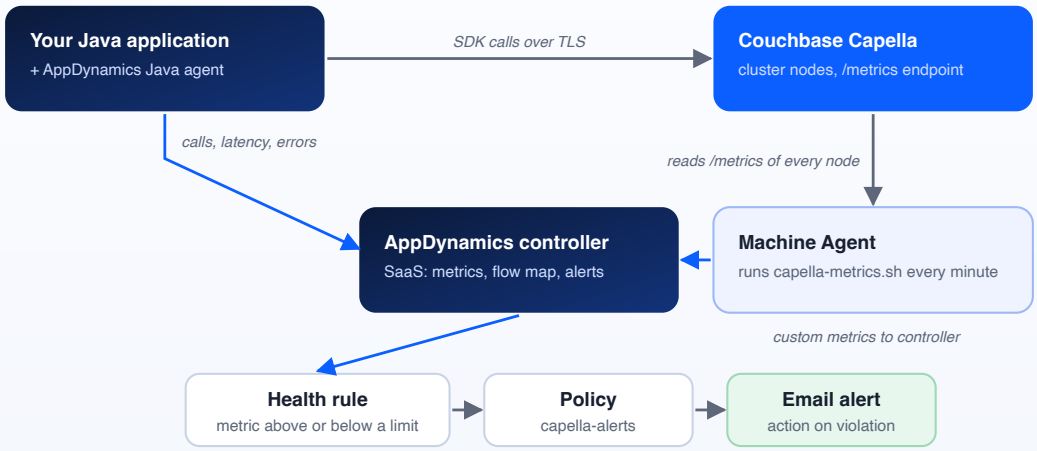
What you will build

1 Application view

Latency, call counts and errors for every call your Java app makes to Capella.

2 Cluster metrics

Any Capella metric you choose (CPU, resident ratio, memory...) as an AppDynamics metric, with an email alert. CPU is the first test.



How the pieces fit together

Capella is managed, so you cannot install an agent on its nodes. The Java agent watches your app's calls. The Machine Agent reads Capella's own metrics endpoint.

Requirements: what you need and what to download

Collect everything in this section first. Nothing here is installed yet.

Accounts and access

YOU NEED	DETAILS
Capella cluster	A running cluster with a bucket, its connection string (`couchbases://cb.xxxx.cloud.couchbase.com`), and a database user and password. The public IP of every host in this guide must be on the cluster's **allowed IP list** (Capella: cluster, Settings).
Metrics user	A database user that can read cluster metrics. If you get `401` or `403` in Part 3, use a user with a monitoring-capable role.
AppDynamics SaaS account	Controller URL (`YOURNAME.saas.appdynamics.com`), plus the **Account Name** and **Account Access Key**. In the controller click the gear icon, then License, then the Account section. A login to the AppDynamics Downloads portal is needed for the agents below.
Java application	Java 8 or newer, using the Couchbase Java SDK 3.x. Any app that talks to Capella works; a plain `main()` program needs the extra step in Part 2.2.
Hosts	The host that runs your app, and a Linux or macOS host for the Machine Agent (it can be the same machine).

Downloads

WHAT	WHERE TO GET IT	USED IN
AppDynamics Java Agent (ZIP)	AppDynamics Downloads portal. Sign in, set **Type** to *Java Agent*, pick the latest version, download the ZIP. (The Getting Started wizard in the controller also offers it.)	Part 2
AppDynamics Machine Agent (ZIP)	Same portal. Set **Type** to *Machine Agent*, filter **Operating System** to yours, download the ZIP. Choose the bundle that includes a JRE if one is listed; otherwise the plain ZIP needs Java 8 or newer on that host.	Part 3

Capella root certificate	Capella UI (Connect section), or extract it with the command in Part 1.1.	Parts 1, 3
Couchbase Java SDK 3.x	Maven Central. Your build tool downloads it from the dependency in Part 1.3.	Part 1
JDK 8 or newer	Only if not already installed, for example Eclipse Temurin from adoptium.net.	Parts 1 to 3
Maven or Gradle	Only if you build the application yourself (maven.apache.org).	Part 1

Command-line tools used in the steps: `curl`, `openssl`, `unzip`, `awk`, `grep`, `sed`. They are installed by default on most Linux and macOS systems.

Network access (outbound)

FROM	TO
Application host	Capella on TCP 11207 and 18091 to 18096 (TLS)
Machine Agent host	Capella on TCP 18091 (metrics) and 18093 (query service)
Both hosts	The AppDynamics controller on TCP 443

Write these values down

VALUE	EXAMPLE
Capella connection string	`couchbases://cb.xxxx.cloud.couchbase.com`
Database user and password	`metrics-user` / (password)
Controller host (bare hostname, no https://)	`YOURNAME.saas.appdynamics.com`
Account Name and Access Key	from the License page
Application, tier and node names	`capella-demo`, `capella-client`, `node1`. Use the same three in both agents.

Suggested folder layout

FOLDERS

```
/opt/appdynamics/java-agent/javaagent.jar                Java agent (Part 2)
/opt/appdynamics/machine-agent/machineagent.jar          Machine Agent (Part 3)
/opt/appdynamics/machine-agent/capella-root.pem          Capella root certificate
/opt/appdynamics/machine-agent/monitors/CapellaMetrics/  monitor.xml, capella-metrics.sh,
metrics.conf, capella.env
```

? Key terms in two minutes

TERM	WHAT IT MEANS HERE
Application	The top-level container for one system you monitor. We use `capella-demo`. Set with `applicationName`.
Tier	A group of nodes that do the same job. We use `capella-client` for your Java app. Set with `tierName`.
Node	One monitored running process: one instance of your app, or one Machine Agent. We use `node1`. Set with `nodeName`. Both agents use the same application, tier and node names so their data lands together.
Business transaction	One unit of work, such as a web request or your `doWork` method. Couchbase calls are recorded inside it.
Backend	An external system your app calls. Capella appears as a Couchbase backend on the flow map.
Custom metric	A number you report yourself. Here: any Capella metric, such as node CPU or resident ratio, reported by the Machine Agent.
Health rule, action, policy	The rule watches a metric, the action does something (sends an email), the policy connects them.

! **Two different "nodes".** A *Capella node* is a database server. An *AppDynamics node* is a monitored process, here your app or the Machine Agent. Capella metrics describe the Capella servers, but they are stored under the AppDynamics node `node1`.

1 Connect your app to Capella over TLS

Capella uses a certificate chain rooted in a Couchbase CA that Java and curl do not trust by default.

1.1 Get the root certificate

Download the cluster's root certificate from the Capella UI (Connect section), or extract it from the server as below. Keep a copy next to your app and one at `/opt/appdynamics/machine-agent/capella-root.pem` (Part 3 uses it).

```bash
H=cb.xxxx.cloud.couchbase.com
echo | openssl s_client -connect "$H:18091" -servername "$H" -showcerts 2>/dev/null \
  | awk '/BEGIN CERT/{n++} {if(n)buf[n]=buf[n] $0 "\n"} END{printf "%s", buf[n]}' > capella-root.pem
```

1.2 Test the connection

```bash
curl --cacert capella-root.pem -u "$CB_USER:$CB_PASSWORD" -o /dev/null -w "%{http_code}\n" \
  "https://$H:18091/pools"      # expect 200
```

1.3 Use it in your application

```xml
<dependency>
  <groupId>com.couchbase.client</groupId>
  <artifactId>java-client</artifactId>
  <version>3.7.5</version>
</dependency>
```

```java
ClusterEnvironment env = ClusterEnvironment.builder()
    .securityConfig(SecurityConfig.enableTls(true)
        .trustCertificate(Paths.get("capella-root.pem")))
    .build();
Cluster cluster = Cluster.connect("couchbases://cb.xxxx.cloud.couchbase.com",
    ClusterOptions.clusterOptions(user, password).environment(env));
```

✓ **Done when: your app connects and reads or writes a document.**

2 See your app's Capella calls in AppDynamics

2.1 Attach the Java agent

Unzip the Java Agent ZIP, then start your app with these flags:

```bash
unzip AppServerAgent-*.zip -d /opt/appdynamics/java-agent
```

```bash
java -javaagent:/opt/appdynamics/java-agent/javaagent.jar \
  -Dappdynamics.controller.hostName=YOURNAME.saas.appdynamics.com \
  -Dappdynamics.controller.port=443 \
  -Dappdynamics.controller.ssl.enabled=true \
  -Dappdynamics.agent.accountName=ACCOUNT_NAME \
  -Dappdynamics.agent.accountAccessKey=ACCESS_KEY \
  -Dappdynamics.agent.applicationName=capella-demo \
  -Dappdynamics.agent.tierName=capella-client \
  -Dappdynamics.agent.nodeName=node1 \
  -jar your-app.jar
```

> ! **Common mistakes.** The host must be the bare hostname (no `https://`). The account name and key come from the License page. A `401 Unauthorized` in `java-agent/ver*/logs/*/agent.log` means one of them is wrong.

✓ **Done when: the agent log shows `Registration information received`, and the application appears under Applications in the controller.**

2.2 Standalone programs only: create a business transaction

AppDynamics only records Couchbase calls inside a business transaction. Web applications get one automatically from each request. A plain Java program has no entry point, so the application shows 0 calls until you define one. Put the work in one method:

```java
public static void doWork(Cluster cluster, Collection col, int i) {   // the unit of work
    col.upsert("doc::" + i, JsonObject.create().put("i", i));
    col.get("doc::" + i);
}
```

Then in the controller: **Applications, your app, Configuration, Instrumentation, Transaction Detection, Rules, Add**:

FIELD	VALUE
Rule type	Custom Match Rule
Agent type, entry point	Java, POJO
Name	`capella-doWork`
Scope	Default Scope
Class name	equals your class, for example `demo.CapellaLoad`
Method name	equals `doWork`

Restart the app. Once traffic is flowing, open the business transaction: the **Flow Map** shows your tier pointing to Couchbase backends, with calls per minute, response time and errors.

✓ **Done when: the Flow Map shows a Couchbase backend.**

3 Report Capella metrics to AppDynamics

Capella publishes its server metrics on a Prometheus-format endpoint. A small script on a Machine Agent reads it every minute and reports the numbers you choose to AppDynamics as **custom metrics**. Which numbers is decided by one list, `metrics.conf`. **CPU is the first entry, used to prove the setup works.** Part 6 shows how to add any other metric by adding a line.

3.1 Check that the metrics endpoint works

BASH

```
curl -s --cacert capella-root.pem -u "$CB_USER:$CB_PASSWORD" "https://$H:18091/metrics" \
  | grep '^sys_cpu_utilization_rate'
# sys_cpu_utilization_rate{category="system",instance="ns_server",job="general"} 8.6
```

Every line in this output is a metric you can report. The number at the end of the line is its value. Your public hostname answers from **one node only**; the script below reads every node.

3.2 Install the Machine Agent and the Capella monitor

BASH

```
unzip MachineAgent-*.zip -d /opt/appdynamics/machine-agent
mkdir -p /opt/appdynamics/machine-agent/monitors/CapellaMetrics
# save monitor.xml, capella-metrics.sh and metrics.conf (below) in that folder, then:
chmod +x /opt/appdynamics/machine-agent/monitors/CapellaMetrics/capella-metrics.sh
```

monitor.xml tells the Machine Agent to run the script every 60 seconds:

```xml
<monitor>
  <name>CapellaMetrics</name>
  <type>managed</type>
  <description>Reports Capella metrics from the Prometheus /metrics endpoint</description>
  <monitor-configuration></monitor-configuration>
  <monitor-run-task>
    <execution-style>periodic</execution-style>
    <execution-frequency-in-seconds>60</execution-frequency-in-seconds>
    <name>Capella Metrics</name>
    <display-name>Capella Metrics</display-name>
    <description>Capella metrics</description>
    <type>executable</type>
    <execution-timeout-in-secs>60</execution-timeout-in-secs>
    <task-arguments></task-arguments>
    <executable-task>
      <type>file</type>
      <file os-type="linux">capella-metrics.sh</file>
      <file os-type="mac">capella-metrics.sh</file>
    </executable-task>
  </monitor-run-task>
</monitor>
```

metrics.conf is the list of metrics to report, one per line (format explained in Part 6). It starts with CPU:

```config
# Metrics to report to AppDynamics. One per line:
#   Display name | Prometheus metric name | how to combine nodes: max, min, avg or sum | optional filter
# Lines starting with # are ignored.
CPU Utilization %|sys_cpu_utilization_rate|max
```

capella-metrics.sh lists every node, downloads each node's metrics and prints one custom-metric line for every entry in `metrics.conf`:

```bash
#!/usr/bin/env bash
# AppDynamics Machine Agent script monitor: reports Capella metrics as custom metrics.
# Which metrics to report is listed in metrics.conf (one line per metric).
# The Machine Agent runs scripts with an empty environment, so credentials come from capella.env
# (written by your start script, deleted on exit).
DIR="$(cd "$(dirname "$0")" && pwd)"
[ -f "$DIR/capella.env" ] && . "$DIR/capella.env"
H="${CB_CONNSTR#couchbases://}"; H="${H%%/*}"
CA="${CAPELLA_CA_PEM:-$DIR/../../../capella-root.pem}"
CONF="$DIR/metrics.conf"
[ -f "$CA" ] && [ -f "$CONF" ] || exit 0
CACHE="$DIR/capella.nodes"   # node list cache, used when the query service is too busy to answer
```

```
# 1) node list: from system:nodes, fall back to the cached list, then to the public hostname alone
RESP=$(curl -sS --max-time 15 --cacert "$CA" -u "$CB_USER:$CB_PASSWORD" \
  --data-urlencode 'statement=SELECT name FROM system:nodes' "https://$H:18093/query/service"
2>/dev/null)
NODES=$(printf '%s' "$RESP" | grep -o '"name": *"[^"]*"' | sed -E 's/"name": *"([^":]*)(:[0-
9]+)?"/\1/' | sort -u)
if [ -n "$NODES" ]; then printf '%s\n' "$NODES" > "$CACHE"; else NODES=$(cat "$CACHE"
2>/dev/null); fi
[ -n "$NODES" ] || NODES="$H"

# 2) download /metrics from every node once
TMP=$(mktemp -d); trap 'rm -rf "$TMP"' EXIT
i=0
for N in $NODES; do
  curl -sS --max-time 15 --cacert "$CA" -u "$CB_USER:$CB_PASSWORD" "https://$N:18091/metrics" -o
"$TMP/$i" 2>/dev/null
  i=$((i+1))
done

# 3) one output line per metric in metrics.conf:  Display name | metric name | max/min/avg/sum |
optional filter
grep -v '^[[:space:]]*#' "$CONF" | while IFS='|' read -r LABEL METRIC AGG FILTER; do
  [ -n "$LABEL" ] && [ -n "$METRIC" ] || continue
  VAL=$(cat "$TMP"/* 2>/dev/null | awk -v m="$METRIC" -v agg="${AGG:-max}" -v f="$FILTER" '
    index($0, m) == 1 && substr($0, length(m) + 1, 1) ~ /[{ ]/ && (f == "" || index($0, f) > 0) {
      v = $NF + 0; n++; sum += v
      if (n == 1 || v > mx) mx = v
      if (n == 1 || v < mn) mn = v
    }
    END { if (n == 0) exit
          r = (agg == "min") ? mn : (agg == "sum") ? sum : (agg == "avg") ? sum / n : mx
          printf "%.0f", r }')
  [ -n "$VAL" ] && echo "name=Custom
Metrics|Capella|$LABEL,value=$VAL,aggregator=OBSERVATION,time-rollup=CURRENT,cluster-
rollup=INDIVIDUAL"
done
exit 0
```

3.3 Give the script its credentials

The Machine Agent runs scripts with an **empty environment**, so exported variables are not visible. Put the credentials in a file next to the script and restrict it:

```
ENV
CB_CONNSTR='couchbases://cb.xxxx.cloud.couchbase.com'
CB_USER='metrics-user'
CB_PASSWORD='the-password'
CAPELLA_CA_PEM='/opt/appdynamics/machine-agent/capella-root.pem'
```

```bash
chmod 600 /opt/appdynamics/machine-agent/monitors/CapellaMetrics/capella.env
```

3.4 Test the script, then start the Machine Agent

```bash
/opt/appdynamics/machine-agent/monitors/CapellaMetrics/capella-metrics.sh
# name=Custom Metrics|Capella|CPU Utilization %,value=9,aggregator=OBSERVATION,time-rollup=CURRENT,cluster-rollup=INDIVIDUAL
```

Start the Machine Agent. Use the **same application, tier and node names** as the Java agent so the metrics land under your application:

```bash
java -Dappdynamics.controller.hostName=YOURNAME.saas.appdynamics.com \
  -Dappdynamics.controller.port=443 -Dappdynamics.controller.ssl.enabled=true \
  -Dappdynamics.agent.accountName=ACCOUNT_NAME \
  -Dappdynamics.agent.accountAccessKey=ACCESS_KEY \
  -Dappdynamics.agent.applicationName=capella-demo \
  -Dappdynamics.agent.tierName=capella-client \
  -Dappdynamics.agent.nodeName=node1 \
  -Dappdynamics.sim.enabled=false \
  -jar /opt/appdynamics/machine-agent/machineagent.jar
```

Open **Metric Browser** (left menu, More). Every line of `metrics.conf` appears here, under `Custom Metrics | Capella`:

`Application Infrastructure Performance | capella-client | Individual Nodes | node1 | Custom Metrics | Capella | CPU Utilization %`

✓ **Done when: the graph shows a CPU value.**

4 Alert on a metric

Three objects work together: an **action** (what to do), a **health rule** (what to watch) and a **policy** (connects them). Everything is under **Alert & Respond** in the left menu of your application. The steps are the same for every metric; we use **CPU above 50%** as the worked example.

4.1 Turn on email sending

1. Open **Alert & Respond, Email / SMS Configuration**.
2. Tick **Enable Email / SMS Notifications**.

3. Keep the default SMTP host and port. Enter a **From Address** (for example your own address). Leave authentication off.
4. Under **Test SMTP Configuration**, enter your address and click **Send**. Confirm the test mail arrives, then click **Save**.

4.2 Create the email action

1. Open **Alert & Respond, Actions**, then click **Create**.
2. Choose **Send an Email** and click OK.
3. Name it `email-me`. Enter the recipients in **To** (add only the people who should be alerted). Optionally add a note.
4. **Save.** The action appears in the list.

4.3 Create a health rule for a metric

1. Open **Alert & Respond, Health Rules**, then click **+**.
2. **Set the rule type first** (Affected Entities tab), because changing it later resets the conditions you entered.

TAB	SETTING
Overview	A name that says what it watches, for example `Capella CPU > 50`. Enabled ticked. Use data from last **5** min. Wait time after violation **5** min.
Affected Entities	Health rule type: **Tier / Node Health - Hardware, JVM, CLR**. What does it affect: **Nodes**. Choose **These specified Nodes** and move your node (for example `node1`) into Selected.
Critical Criteria	Click **Add Condition**. Set **Single Metric**, **Value**. Click **Select a Metric**, choose **Specify a Relative Metric Path** and type `Custom Metrics|Capella|<Display name>`, using the display name from `metrics.conf` (for CPU: `Custom Metrics|Capella|CPU Utilization %`). Then **Select Metric**. Choose the comparison and value (see the table below). Leave "Trigger only when violation occurs" unticked. Keep **Any Node**. "No data" can stay on **Unknown**.
Warning Criteria	Optional. Repeat with a milder value for an early warning.

Click **Save** (bottom right of the dialog). Examples of conditions, one rule each:

WHAT TO WATCH	DISPLAY NAME IN METRICS.CONF	COMPARISON	EXAMPLE VALUE
Node CPU too high	`CPU Utilization %`	> Specific Value	50
Data not in memory	`Resident Ratio %`	< Specific Value	90

Any other metric	the name you gave it	> or <	your limit

Nodes or Tiers? Choose **Nodes**, because the custom metrics are stored under the node (`Individual Nodes | node1 | Custom Metrics | Capella` in Metric Browser). Choose **Tiers** only for metrics stored at tier level, and then use **The Tier Average** instead of Any Node. If a rule never fires, check that the entity you picked is where Metric Browser shows the metric.

The metric tree in this dialog lists only the standard hardware metrics. Your custom metrics do not appear in it, which is why you use the relative path.

4.4 Create the policy (once, for all rules)

1. Open **Alert & Respond, Policies**, then click **+**.
2. **Trigger tab:** name it `capella-alerts`, keep Enabled ticked, expand **Health Rule Violation Events** and tick **Started - Warning** and **Started - Critical**.
3. **Health Rule Scope tab:** choose **These Health Rules**, click **+** and add your rule (for example `Capella CPU > 50`). Add every new rule here later.
4. **Object Scope tab:** leave the default (any object).
5. **Actions tab:** click **+** and choose `email-me`.
6. **Save.** The policy lists your rules and your action.

✓ **Done when: the policy is saved, the rule is enabled, and the email test in 4.1 arrived.**

4.5 Optional: alert on your app's own errors and latency

The same three objects work for the application view from Part 2. Create two more health rules the same way (4.3), with type **Business Transaction Performance**, affecting your business transaction (for example `capella-doWork`), then add both to the policy under Health Rule Scope.

RULE NAME	CRITICAL CONDITION
`capella-doWork errors`	**Errors per Minute** > Specific Value **1**
`capella-doWork slow`	**Average Response Time (ms)** > Specific Value **300** (adjust to your normal latency)

Errors that your code catches itself are not counted. AppDynamics counts an error when an exception leaves the business transaction method.

5 Test the alert

Test with CPU, because it is the easiest metric to push up. The same method works for any metric.

1. **Quick test first.** Temporarily set the rule value to **5**. Normal idle CPU is above that, so the alert fires without any load. For faster feedback you can also set the data window and wait time to 1 minute while testing, then return them to 5. For a metric where lower is worse, set the limit on the other side of its current value instead.
2. **Then generate real load.** Make sure the Machine Agent is running, then run the load script. It runs CPU-heavy queries and reads or writes no data. It slows other queries on that cluster while it runs, so use a test cluster or a quiet time:

```bash
# Optional load test. Starts 24 parallel CPU-heavy queries for 5 minutes (no data is read or written).
for i in $(seq 1 24); do
  ( while true; do
      curl -s --cacert capella-root.pem -u "$CB_USER:$CB_PASSWORD" \
        --data-urlencode 'statement=SELECT SUM(x*x) FROM ARRAY_RANGE(0,3000000) AS x' \
        "https://$H:18093/query/service" > /dev/null
    done ) &
done
sleep 300; kill $(jobs -p)
```

If CPU stays low, increase the number of parallel loops.

3. Watch **Metric Browser** (the metric moves), then **Alert & Respond, Health Rules** or the application's **Events** for *Health Rule Violation Started - Critical*, then your inbox.
4. Set the value back to your real limit (for CPU, **50**).

> **One alert per violation.** While the metric stays past the threshold the violation stays open, and AppDynamics does not send a new *Started* alert. If you re-test, stop the load, wait for the metric to recover and the violation to clear (the rule returns to healthy), then run the load again.

6 Add more metrics

CPU was the test. To report any other Capella metric, you only edit `metrics.conf`. No restart is needed: the script reads the file every minute.

1. **Find the metric name.** Search the endpoint output by keyword, for example `resident`, `memory` or `disk`:

```bash
# 1. Find the metric you want. Search the endpoint output by keyword, for example "resident":
curl -s --cacert capella-root.pem -u "$CB_USER:$CB_PASSWORD" "https://$H:18091/metrics" \
  | grep -i resident | grep -v '^#' | head
```

2. **Add one line** to `metrics.conf` with four fields separated by `|`:

FIELD	MEANING
1. Display name	The name you will see in AppDynamics and use in the health rule, for example `Resident Ratio %`.
2. Metric name	The exact name from step 1, for example the text before `{` on the line you found.
3. Combine nodes	`max` (worst node when high is bad, like CPU), `min` (worst node when low is bad, like a ratio), `avg` or `sum`.
4. Filter (optional)	Text that must appear on the line, to report one bucket only, for example `bucket="travel-sample"`.

```config
# Display name | metric name | combine nodes | optional filter
CPU Utilization %|sys_cpu_utilization_rate|max
Resident Ratio %|REPLACE_WITH_METRIC_NAME|min
Resident Ratio travel-sample %|REPLACE_WITH_METRIC_NAME|min|bucket="travel-sample"
```

3. **Check it.** After a minute the new display name appears in Metric Browser under `Custom Metrics | Capella`.
4. **Alert on it.** Create a health rule as in 4.3 using that display name in the relative path, choose the comparison, and add the rule to the `capella-alerts` policy (4.4).

> The metric name must match exactly. If a line produces nothing, run the search again and copy the name from the output. Metrics that exist once per bucket carry a `bucket` label on every line; without a filter the script combines all buckets into one value.

Troubleshooting

SYMPTOM	CAUSE	FIX
curl or Java: *self signed certificate in certificate chain*	The Capella root CA is not trusted by default.	Use `--cacert capella-root.pem` and the SDK `trustCertificate` setting.

Agent log: *401 Unauthorized*	Wrong account name or access key.	Copy both from the controller: gear, License, Account. Re-type the key; check for stray spaces.
Agent log shows `hostName=https://...`	Scheme included in the controller host.	Use the bare hostname, no `https://`.
Application visible, 0 calls, no Couchbase backend	A plain Java program has no business transaction entry point.	Add the POJO rule in Part 2 (not needed for web apps).
Metrics never appear	The monitor script runs with an empty environment, so it cannot see exported variables.	Put the credentials in `capella.env` next to the script (Part 3).
One metric is missing	The metric name in `metrics.conf` does not match exactly.	Search the endpoint again (Part 6) and copy the name from the output.
A value in AppDynamics is lower or higher than Capella shows	The public hostname answers `/metrics` from one node only.	Use the script in this guide: it lists every node and combines them with max, min, avg or sum.
Health rule never fires	Wrong entity or metric path.	Affect the node, choose the metric by relative path, and set "no data" to Critical while testing.
Alert emailed once, then nothing more	A violation stays open while CPU stays high, and the policy triggers on *Started* only.	Expected. To hear again, add *Violation Continues* to the policy trigger, or let the violation clear and re-test.
Alert emails go to the wrong people	Extra recipients on the email action.	Edit the action under Alert & Respond, Actions.

i Good to know

- **Security.** Keep the DB password and AppDynamics key out of source control. `capella.env` must be readable only by the Machine Agent user. Rotate any key that was shown on a screen or shared.
- **Scope.** This alerts on metrics read from Capella's metrics endpoint, with limits you choose. Capella also has its own built-in alerts with fixed levels (see Capella's Alert Reference).
- **Not covered.** Containers and Kubernetes, and dashboards.
- **macOS only.** The Machine Agent's built-in Sigar hardware monitor may trigger a Gatekeeper prompt. Click Done and disable that monitor (`monitors/JavaHardwareMonitor/monitor.xml`, set `enabled` to

`false`). Not needed on Linux.

Capella + AppDynamics setup guide
Written for Java agent 26.9, Machine Agent 26.8 and Couchbase Java SDK 3.7.5. Menu labels in the AppDynamics UI can differ slightly between versions.